

✓ Aprendizado Profundo - UFMG

Preâmbulo

O código abaixo consiste dos imports comuns. Além do mais, configuramos as imagens para ficar de um tamanho aceitável e criamos algumas funções auxiliares. No geral, você pode ignorar a próxima célula.

```
# -*- coding: utf8

import matplotlib.pyplot as plt
import torch
import numpy as np

plt.rcParams['figure.figsize'] = (18, 10)
plt.rcParams['axes.labelsize'] = 20
plt.rcParams['axes.titlesize'] = 20
plt.rcParams['legend.fontsize'] = 20
plt.rcParams['xtick.labelsize'] = 20
plt.rcParams['ytick.labelsize'] = 20
plt.rcParams['lines.linewidth'] = 4
torch.set_printoptions(precision=10)
```

```
plt.ion()

plt.rcParams['figure.figsize'] = (12, 8)
```

Para testar o resultado dos seus algoritmos vamos usar o módulo testing do numpy.

```
from numpy.testing import assert_equal
from numpy.testing import assert_almost_equal
from numpy.testing import assert_array_almost_equal
```

Regressão Linear e Logística from Scratch

Para brincar um pouco mais com essa diferenciação automática, presente em frameworks como pytorch e mxnet, nesta aula vamos implementar a regressão linear e logística do zero. Vamos fazer duas versões de cada:

1. Derivando na mão, não é complicado.
2. Derivando com autograd de pytorch

✓ Conjunto de Problemas 1: Mais Derivadas

Antes de entrar na regressão, vamos brincar um pouco de derivadas dentro de funções. Dado dois números x e y , implemente a função `log_exp` que retorna:

$$-\log\left(\frac{e^x}{e^x + e^y}\right)$$

```
def log_exp(x, y):
    return -torch.log(torch.exp(x) / (torch.exp(x) + torch.exp(y)))
```

1. Abaixo vamos testar o seu código com algumas entradas simples.

```
x, y = torch.tensor([2.0]), torch.tensor([3.0])
z = log_exp(x, y)
z
```

```
tensor([1.3132617474])
```

```
# Teste. Não apague
assert_almost_equal(1.31326175, z.numpy())
```

2. A função a seguir computa $\partial z / \partial x$ e $\partial z / \partial y$ usando `autograd`.

```
# O argumento funcao_forward é uma função python. Será a sua log_exp.
# A ideia aqui é deixar claro a ideia de forward e backward propagation, depois
# de avaliar a função chamamos backward e temos as derivadas.
def grad(funcao_forward, x, y):
    x.requires_grad_(True)
    y.requires_grad_(True)
    z = funcao_forward(x, y)
    z.backward()
    return x.grad, y.grad
```

Testando

```
x, y = torch.tensor([2.0], dtype = torch.double), torch.tensor([3.0], dtype = torch.double)
dx, dy = grad(log_exp, x, y)
```

```
assert_almost_equal(-0.7310586, dx.numpy())
assert_almost_equal(0.7310586, dy.numpy())
```

4. Agora teste com números maiores, algum problema?

```
x, y = torch.tensor([400.0]).double(), torch.tensor([800.0]).double()
grad(log_exp, x, y)
```

```
(tensor([nan], dtype=torch.float64), tensor([nan], dtype=torch.float64))
```

5. Pense um pouco sobre o motivo do erro acima. Usando as propriedades de logaritmos, é possível fazer uma função mais estável. Abaixo segue a implementação da mesma. O problema aqui é que o exponencial "explode" quando x ou y são muito grandes. Este [link](#) pode ajudar.

```
x, y = torch.tensor([400.0], dtype = torch.double), torch.tensor([800.0], dtype = torch.double)
def stable_log_exp(x, y):
    return torch.log(torch.exp(y-x))

dx, dy = grad(stable_log_exp, x, y)
```

```
stable_log_exp(x, y)
```

```
tensor([400.], dtype=torch.float64, grad_fn=<LogBackward0>)
```

```
# Teste. Não apague
assert_equal(-1, dx.numpy())
assert_equal(1, dy.numpy())
```

O exemplo acima mostra um pouco de problemas de estabilidade numérica. Às vezes é melhor usar versões alternativas de funções. Isto vai ocorrer quando você ver vários nans na sua frente :-). Claro, estamos assumindo que existe uma outra função equivalente que é mais estável para o computador.

✓ Conjunto de Problemas 2: Regressão Linear

Agora, vamos explorar uma regressão linear. Embora não vamos fazer uso da logexp acima, a ideia de derivar parcialmente dentro de funções pode nos ajudar. Lembrando da regressão linear, inicialmente temos um conjunto de observações representadas como tuplas $(\mathbf{x}_i, y_i)$. Aqui, $\mathbf{x}_i$ é um vetor de atributos. Vamos forçar $\mathbf{x}_{i0} = 1$, capturando assim o intercepto. Além do mais, $\mathbf{x}_{ij}$ quando $j \neq 0$, são os outros atributos de entrada. y_i um valor real representando uma resposta. Nossa regressão visa capturar:

$$y_i = 1 + \theta_1 x_{i1} + \theta_2 x_{i2} + \dots + \theta_k x_{if}$$

Lembrando da regressão linear multivariada, podemos representar as equações como uma multiplicação de uma matriz com um vetor

7. Crie uma função de previsão. A mesma recebe uma matriz $\mathbf{X}$ e um vetor de parâmetros θ . Sua função deve retornar um vetor de previsões para cada linha de $\mathbf{X}$. Não use nenhum laço!!

```
def previsao(X, theta):
    return torch.matmul(X, theta)
```

Erros quadrados. Para aprender os parâmetros ótimos da regressão linear, precisamos fazer uso de um modelo de erros quadrados. Em particular nosso objetivo é aprender os parâmetros que minimizam a função:

$$L(\theta) = n^{-1} \sum_i (\hat{y}_i - y_i)^2$$

Onde $\hat{y}_i$ é uma previsão (vêm da sua função python acima). y_i é o valor real dos dados.

8. Implemente uma função para a média dos os erros quadrados.

```
def media_erros_quadrados(X, theta, y):  
    y_hat = previsao(X, theta).squeeze() # Squeeze garante alinhamento de dimensões  
    return torch.mean((y_hat - y) ** 2)
```

9. Agora, crie uma função que deriva o erro acima. A função deve usar o autograd do pytorch (função backward). Lembre-se que temos um vetor de parâmetros θ . Por sorte, você pode fazer derivadas de tais vetores também.

```
def derivada_torch(X, theta, y):  
    # Lembre que o theta passado devera ser uma variavel com requires_grad == True  
    loss = media_erros_quadrados(X, theta, y)  
    loss.backward()  
    return theta.grad
```

10. A versão a seguir não usa autograd. As derivadas são implementadas do zero.

```
def derivada_naveria(X, theta, y):  
    return ((torch.matmul(X, theta) - y) * X.T).mean(axis=1)
```

11. Por fim, otimize sua função usando o algoritmo de gradiente descendente abaixo.

```
def gd(d_fun, loss_fun, X, y, lambda_=0.01, tol=0.00001, max_iter=10000):
    '''
    Executa Gradiente Descendente. Aqui:

    Parâmetros
    -----
    d_fun : é uma função de derivadas
    loss_fun : é uma função de perda
    X : é um vetor de fatores explanatórios.
        Copie seu código de intercepto da primeira aula.
        para adicionar o intercepto em X.
    y : é a resposta
    lambda : é a taxa de aprendizagem
    tol : é a tolerância, define quando o algoritmo vai parar.
    max_iter : é a segunda forma de parada, mesmo sem convergir
        paramos depois de max_iter iterações.
    '''
    theta = torch.randn(X.shape[1])
    theta.requires_grad_(True)

    print('Iter {}'; theta = '.format(0), theta)

    old_err_sq = np.inf
    i = 0
    while True:
        # Computar as derivadas

        theta.requires_grad_(True) #Necessario pois ao final, theta recebe o theta novo, que não tem requires_grad == True
        grad = d_fun(X, theta, y)

        # Atualizar
        with torch.no_grad():
            theta_novo = theta - lambda_ * grad

        #Parar quando o erro convergir
        err_sq = loss_fun(X, theta, y)
        if torch.abs(old_err_sq - err_sq) <= tol:
            break

        #Atualizar parâmetros e erro
        theta = theta_novo
        old_err_sq = err_sq

        # Informação de debug
        print('Iter {}'; theta = '.format(i+1), theta)
        i += 1
        if i == max_iter:
            break

    return theta
```

```
# Para testes, não apague!!!
X = torch.zeros(1000, 2)
X[:, 0] = 1.0
X[:, 1] = torch.randn(1000)

theta_0_real = 7.0
theta_1_real = 9.0
y = theta_0_real + theta_1_real * X[:, 1]
```

```
# Testando com derivada autograd. Sua função deve retornar algo perto de 7 e 9
theta = gd(derivada_torch, media_erros_quadrados, X, y)
```

```
Iter 323; theta = tensor([6.9893000041, 8.9081997299])
Iter 324; theta = tensor([6.9897718430, 8.9687738419])
Iter 325; theta = tensor([6.9899787903, 8.9693374634])
Iter 326; theta = tensor([6.9901814461, 8.9698915482])
Iter 327; theta = tensor([6.9903798103, 8.9704351425])
Iter 328; theta = tensor([6.9905743599, 8.9709692001])
Iter 329; theta = tensor([6.9907650948, 8.9714937210])
Iter 330; theta = tensor([6.9909520149, 8.9720087051])
Iter 331; theta = tensor([6.9911351204, 8.9725141525])
Iter 332; theta = tensor([6.9913144112, 8.9730110168])
Iter 333; theta = tensor([6.9914898872, 8.9734983444])
Iter 334; theta = tensor([6.9916620255, 8.9739770889])
Iter 335; theta = tensor([6.9918308258, 8.9744472504])
Iter 336; theta = tensor([6.9919958115, 8.9749088287])
Iter 337; theta = tensor([6.9921574593, 8.9753618240])
Iter 338; theta = tensor([6.9923162460, 8.9758071899])
Iter 339; theta = tensor([6.9924716949, 8.9762439728])
Iter 340; theta = tensor([6.9926238060, 8.9766731262])
Iter 341; theta = tensor([6.9927730560, 8.9770946503])
Iter 342; theta = tensor([6.9929194450, 8.9775085449])
Iter 343; theta = tensor([6.9930624962, 8.9779148102])
Iter 344; theta = tensor([6.9932026863, 8.9783134460])
Iter 345; theta = tensor([6.9933400154, 8.9787054062])
Iter 346; theta = tensor([6.9934744835, 8.9790897369])
Iter 347; theta = tensor([6.9936065674, 8.9794673920])
Iter 348; theta = tensor([6.9937357903, 8.9798383713])
Iter 349; theta = tensor([6.9938626289, 8.9802026749])
Iter 350; theta = tensor([6.9939866066, 8.9805603027])
Iter 351; theta = tensor([6.9941082001, 8.9809112549])
Iter 352; theta = tensor([6.9942274094, 8.9812564850])
Iter 353; theta = tensor([6.9943442345, 8.9815950394])
Iter 354; theta = tensor([6.9944586754, 8.9819278717])
Iter 355; theta = tensor([6.9945707321, 8.9822540283])
Iter 356; theta = tensor([6.9946804047, 8.9825744629])
Iter 357; theta = tensor([6.9947881699, 8.9828891754])
Iter 358; theta = tensor([6.9948935509, 8.9831981659])
Iter 359; theta = tensor([6.9949970245, 8.9835014343])
Iter 360; theta = tensor([6.9950981140, 8.9837989807])
```

theta

tensor([6.9950981140, 8.9837989807], requires_grad=True)

```
# Testando com derivada manual. Sua função deve retornar algo perto de 7 e 9
theta = gd(derivada_naverá, media_erros_quadrados, X, y)
```

```

Iter 643; theta = tensor([6.9875102043, 8.9759922020])
Iter 646; theta = tensor([6.9876360893, 8.9762096405])
Iter 647; theta = tensor([6.9877605438, 8.9764242172])
Iter 648; theta = tensor([6.9878835678, 8.9766368866])
Iter 649; theta = tensor([6.9880056381, 8.9768476486])
Iter 650; theta = tensor([6.9881262779, 8.9770565033])
Iter 651; theta = tensor([6.9882459641, 8.9772634506])
Iter 652; theta = tensor([6.9883642197, 8.9774684906])
Iter 653; theta = tensor([6.9884815216, 8.9776725769])
Iter 654; theta = tensor([6.9885973930, 8.9778747559])
Iter 655; theta = tensor([6.9887123108, 8.9780750275])
Iter 656; theta = tensor([6.9888257980, 8.9782733917])
Iter 657; theta = tensor([6.9889383316, 8.9784698486])
Iter 658; theta = tensor([6.9890499115, 8.9786643982])
Iter 659; theta = tensor([6.9891600609, 8.9788570404])

```

theta

tensor([6.9891600609, 8.9788570404], requires_grad=True)

12. Altere a função de Gradiente Descendente para funcionar com minibatches. Em outras palavras, não compute o erro usando todos os dados de X. Use um `slice` de tamanho do minibatch. Uma ideia é seguir o pseudocódigo abaixo.

```

index = np.arange(len(X))
while True:
    minib = np.random.choice(index, minibatchsize) # aqui estou usando numpy para selection minibatch elementos
    X_batch = X[minib]

```

```

# Exemplo abaixo
index = np.arange(len(X))
mb = np.random.choice(index, 50)
print(mb)
X[mb]

```

```

[994 119 416 541 191 794 889 211 885 549 212 284 449 417 877 263 672 631
 395 102  63 855 830 602 412 306  42 820 671 729 652 688 252 274 881 763
 547 342  19 920 815 878 225 964 948 781 216 608 467 715]
tensor([[ 1.0000000000, -1.4977228642],
        [ 1.0000000000, -0.7341976166],
        [ 1.0000000000,  0.0488394611],
        [ 1.0000000000,  0.7101356983],
        [ 1.0000000000, -0.1863205880],
        [ 1.0000000000, -1.9142665863],
        [ 1.0000000000,  0.2244938761],
        [ 1.0000000000, -0.7489461303],
        [ 1.0000000000, -0.4988000095],
        [ 1.0000000000, -1.6106560230],
        [ 1.0000000000, -0.5237780809],
        [ 1.0000000000, -1.6605919600],
        [ 1.0000000000, -0.9096009135],
        [ 1.0000000000, -0.5088320971],
        [ 1.0000000000,  0.5250037313],
        [ 1.0000000000, -0.9192402959],
        [ 1.0000000000, -0.5925052166],
        [ 1.0000000000,  0.4678919017],
        [ 1.0000000000,  1.3057179451],
        [ 1.0000000000,  0.5890341997],
        [ 1.0000000000, -0.9354375005],
        [ 1.0000000000,  1.5207099915],
        [ 1.0000000000, -0.0863608047],
        [ 1.0000000000,  0.7644208670],
        [ 1.0000000000,  0.0971093997],
        [ 1.0000000000,  1.3546854258],
        [ 1.0000000000, -0.1193642095],
        [ 1.0000000000, -0.2710247636],
        [ 1.0000000000, -1.3862020969],
        [ 1.0000000000, -0.2690633833],
        [ 1.0000000000,  1.4266356230],
        [ 1.0000000000,  0.2337200344],
        [ 1.0000000000,  0.6532335281],
        [ 1.0000000000, -2.4850137234],
        [ 1.0000000000,  0.8414285779],
        [ 1.0000000000,  1.7376488447],
        [ 1.0000000000,  0.1191252321],
        [ 1.0000000000, -0.8706756830],
        [ 1.0000000000, -1.1816213131],
        [ 1.0000000000, -0.2818853855],
        [ 1.0000000000,  1.2900421619],
        [ 1.0000000000,  0.9800587296],
        [ 1.0000000000, -1.4851043224],
        [ 1.0000000000, -0.7397115231],
        [ 1.0000000000,  0.7218702435],
        [ 1.0000000000, -1.2336972952],
        [ 1.0000000000, -0.1102656573],
        [ 1.0000000000, -0.9434416890],

```

```
[ 1.0000000000,  0.5749001503],
[ 1.0000000000,  0.9514122009]]])
```

```
def minibatch_gd(d_fun, loss_fun, X, y, lambda_=0.01, tol=0.0001,
                 max_iter=10000, batch_size=10):

    # Inicializando theta com o mesmo tipo do X (ex: double) para evitar incompatibilidade
    theta = torch.randn(X.shape[1], dtype=X.dtype)

    old_err_sq = np.inf
    i = 0
    index = np.arange(len(X))

    while True:
        # Sortear índices do minibatch usando numpy
        minib = np.random.choice(index, batch_size)
        X_batch = X[minib]
        y_batch = y[minib]

        # Computar as derivadas no minibatch
        theta.requires_grad_(True)
        grad = d_fun(X_batch, theta, y_batch)

        # Atualizar
        with torch.no_grad():
            theta_novo = theta - lambda_ * grad

        # Avaliar erro com todos os dados (para manter estabilidade no critério de parada)
        err_sq = loss_fun(X, theta, y)
        if torch.abs(old_err_sq - err_sq) <= tol:
            break

        # Atualizar parâmetros e erro
        theta = theta_novo
        old_err_sq = err_sq

        i += 1
        if i == max_iter:
            break

    return theta
```

```
# Sua função deve retornar algo perto de 7 e 9
minibatch_gd(derivada_torch, media_erros_quadrados, X, y)
```

```
tensor([6.9891629219, 8.9795560837], requires_grad=True)
```

✓ Conjunto de Problemas 3: Logistic from Scratch

12. Repita o mesmo processo para a Logística. Lembrando que a mesma tem a seguinte forma:

$$f(x_i) = \frac{1}{1 + e^{-(1 + \theta_1 x_{i1} + \theta_2 x_{i2} + \dots + \theta_k x_{ik})}}$$

Implemente a função logística.

```
def logistic(X, theta):
    return 1.0 / (1.0 + torch.exp(-torch.matmul(X, theta)))
```

```
# testes, não apague!
X_teste = torch.randn(1000, 20000)
theta = torch.randn(20000)
y_hat_teste = logistic(X_teste, theta)
assert_equal(True, (y_hat_teste >= 0).numpy().all())
assert_equal(True, (y_hat_teste <= 1).numpy().all())
```

Usando a logística acima implemente uma função `logistica_prever` que retorna 0 ou 1. Use o limiar dado na função.

```
def logistica_prever(X, theta, limiar=0.5):
    y_prob = logistic(X, theta)
    return (y_prob >= limiar).double()
```

```
# testes, não apague!
X_teste = torch.randn(1000, 20000)
theta = torch.randn(20000)
y_hat_teste = logistica_prever(X_teste, theta)
```

```
for yi in y_hat_teste.numpy():
    assert(yi in {0, 1})
```

Agora, implemente uma função de entropia cruzada da logística. A mesma, é proporcional ao inverso da verossimilhança. Para entender a derivação entre as duas faça uso dos [Slides](#).

Sendo:

$$l(x_i, y_i | \theta) = y_i \log f(x_i \theta) + (1 - y_i) \log(1 - f(x_i \theta))$$

A verossimilhança para uma observação. A entropia cruzada é a media da negação do termo para todos os exemplos:

$$L(\theta) = -n^{-1} \sum_i ((1 - y_i) \log(1 - f_\theta(x_i)) + y_i \log(f_\theta(x_i)))$$

(dica: use `torch.clamp(logistic(X, theta), min = 0.001, max = 0.999)`). A função remove os 0 e 1s da logistic, evitando assim o valor `log(0)`.

```
def cross_entropy_mean(X, theta, y):
    # 0 squeeze é útil pois X*theta pode virar vetor coluna enquanto y é array 1D no teste
    f_x = torch.clamp(logistic(X, theta), min=0.001, max=0.999).squeeze()
    return -torch.mean((1 - y) * torch.log(1 - f_x) + y * torch.log(f_x))
```

```
# testes, não apague!
from sklearn import datasets
state = np.random.seed(20190187)

X, y = datasets.make_blobs(n_samples=200, n_features=2, centers=2)
X = torch.tensor(X).double()
y = torch.tensor(y).double()

for _ in range(100):
    theta = torch.randn(2,1).double()
    assert(cross_entropy_mean(X, theta, y) >= 0)
```

Agora implemente a derivada.

```
def derivada_torch_logit(X, theta, y):
    loss = cross_entropy_mean(X, theta, y)
    loss.backward()
    return theta.grad
```

A partir daqui basta executar código.

```
plt.scatter(X.numpy()[:, 0], X.numpy()[:, 1], s=80, edgecolors='k')
```



```
<matplotlib.collections.PathCollection at 0x7b4bccdd8a3c0>
```

12

10

13. Assim como foi feito na primeira aula, abaixo testamos o seu código com um toy dataset.

```
# Use essa função antes de executar o GD
def add_intercept(X):
    Xn = torch.zeros(X.shape[0], X.shape[1] + 1).double()
    Xn[:, 0] = 1
```